

Cybersecurity Assignment - Setting Up IDS with Snort and Attack Simulation

Name:-Pruthak Kansara

Date:-5/9/2024

Instructor Name:-Krapesh Bhatt

Table of Contents

- Introduction
- Objectives
- Environment Setup
 - Virtual Machines Creation
- Installing and Configuring OpenSSH on Ubuntu Server
- Installing Snort on Ubuntu Server
- Simulating Attacks with Kali Linux
- Setting Up Dashboard for Snort Alerts
- Testing and Results
- Conclusion

Introduction

In today's ever-evolving cybersecurity landscape, detecting and mitigating threats in real-time is crucial for ensuring the safety of networked systems. One of the most important tools in network security is an Intrusion Detection System (IDS), which helps identify suspicious activity and alerts administrators of potential attacks.

This project aims to set up a small-scale network security lab using Snort IDS to monitor and detect cyber threats. By simulating common attack scenarios, such as port scanning and brute-force attacks, we can observe how Snort detects and reports these threats in real-time. Additionally, a dashboard will be configured to visualise the alerts, offering insights into threat patterns.

The lab environment consists of two virtual machines (VMs): a Kali Linux machine serving as the attack platform and an Ubuntu server running Snort IDS, which will act as the target for the simulated attacks. This hands-on project provides practical experience in deploying an IDS, configuring detection rules, and using real-world attack techniques to understand how IDSs respond to threats.

Objectives

The main goal of this project is to create a network security lab to simulate cyber attacks and monitor them using Snort IDS. By the end of the project, the following objectives will be achieved:

1. Set up Virtual Machines for Kali Linux (attacker) and Ubuntu Server (target with Snort IDS).
2. Install and Configure OpenSSH on the Ubuntu Server to enable brute-force attack testing.
3. Deploy and Configure Snort IDS to detect port scans and brute-force login attempts.
4. Simulate Cyber Attacks from Kali Linux using tools like Nmap (port scan) and Hydra (brute-force).
5. Set up a Dashboard to visualise Snort alerts for real-time threat monitoring.
6. Analyse Detected Threats and document Snort's responses to the simulated attacks.

Environment Setup

To simulate and monitor attacks, two virtual machines (VMs) will be created:

Attacker VM: Kali Linux

- Purpose: Launch network attacks (e.g., port scans, brute-force attempts).
- Setup:
 - Download the Kali Linux ISO from the official website.
 - Create a new VM in VirtualBox or VMware.
 - Allocate sufficient resources (RAM, CPU) and attach the Kali Linux ISO to install the OS.

Target VM: Ubuntu Server

- Purpose: Host Snort IDS and OpenSSH service to detect and monitor attacks.
- Setup:
 - Download the Ubuntu Server ISO.
 - Create a new VM in VirtualBox or VMware.
 - Allocate resources and install Ubuntu Server from the ISO.

Network Configuration:

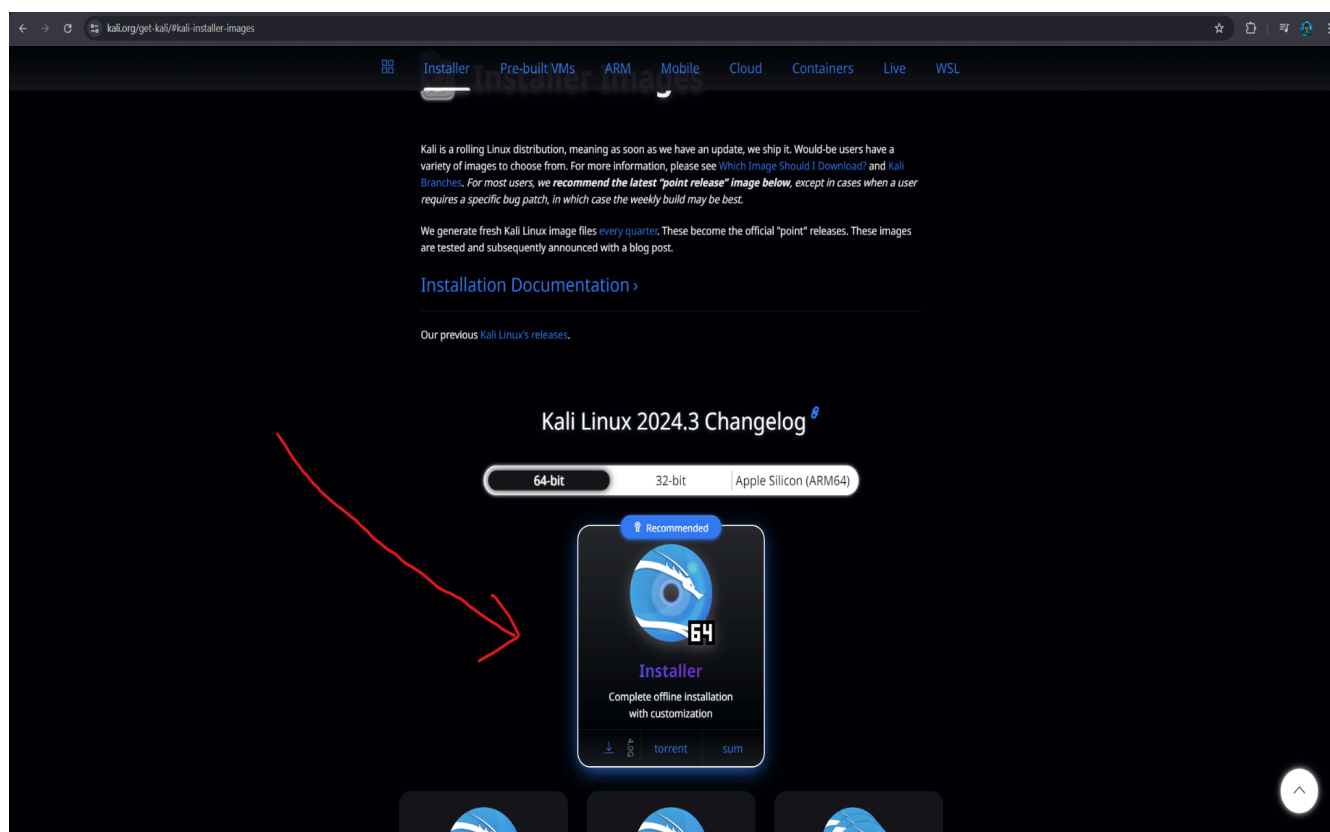
- Both VMs will be on the same internal network to facilitate attack simulation.
- Ensure network connectivity between the two VMs for smooth testing and monitoring

Attacker VM :Kali linux (Setup)

Step 1:-

→ Download the Kali Linux image from:

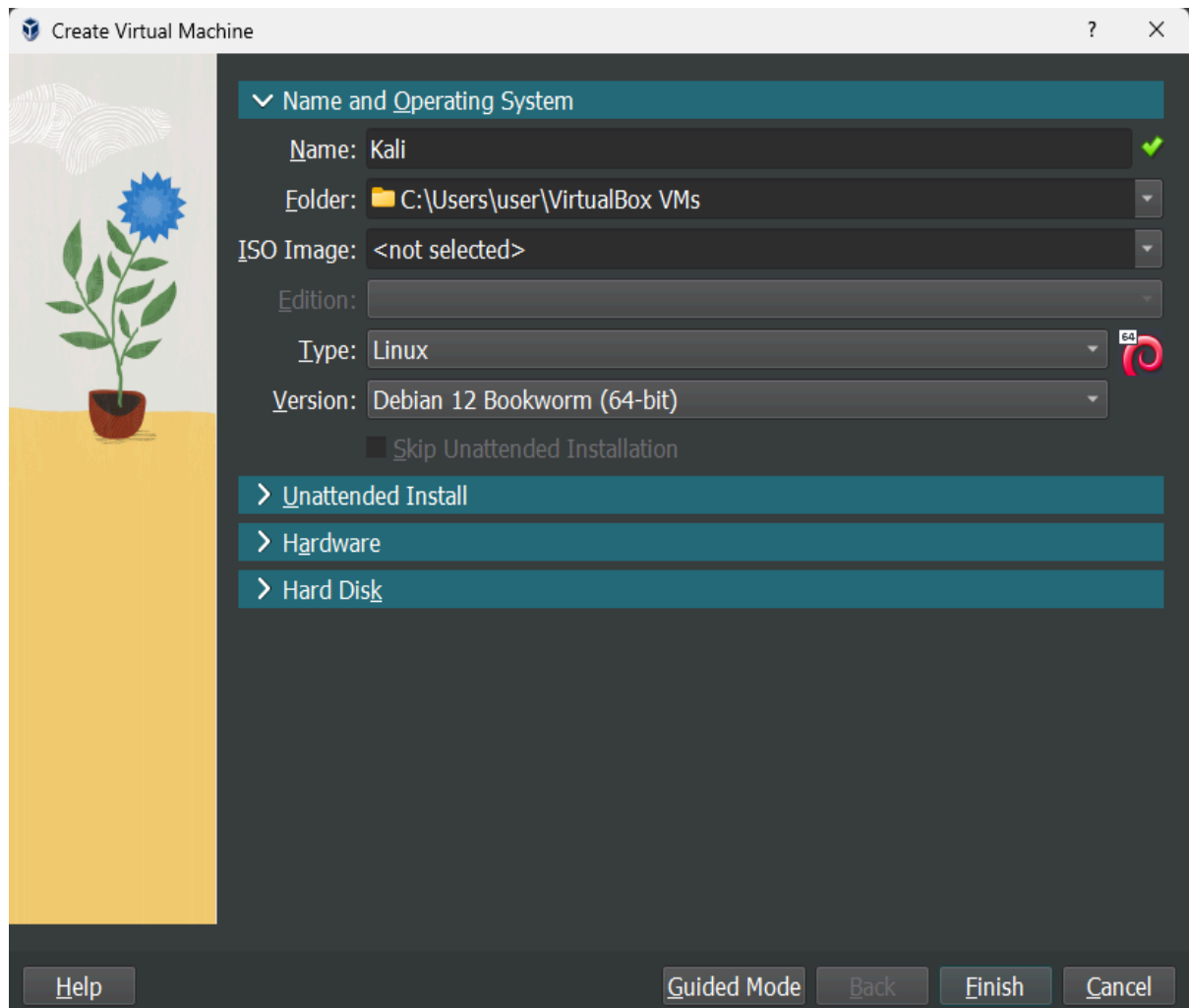
<https://www.kali.org/get-kali/#kali-installer-images>



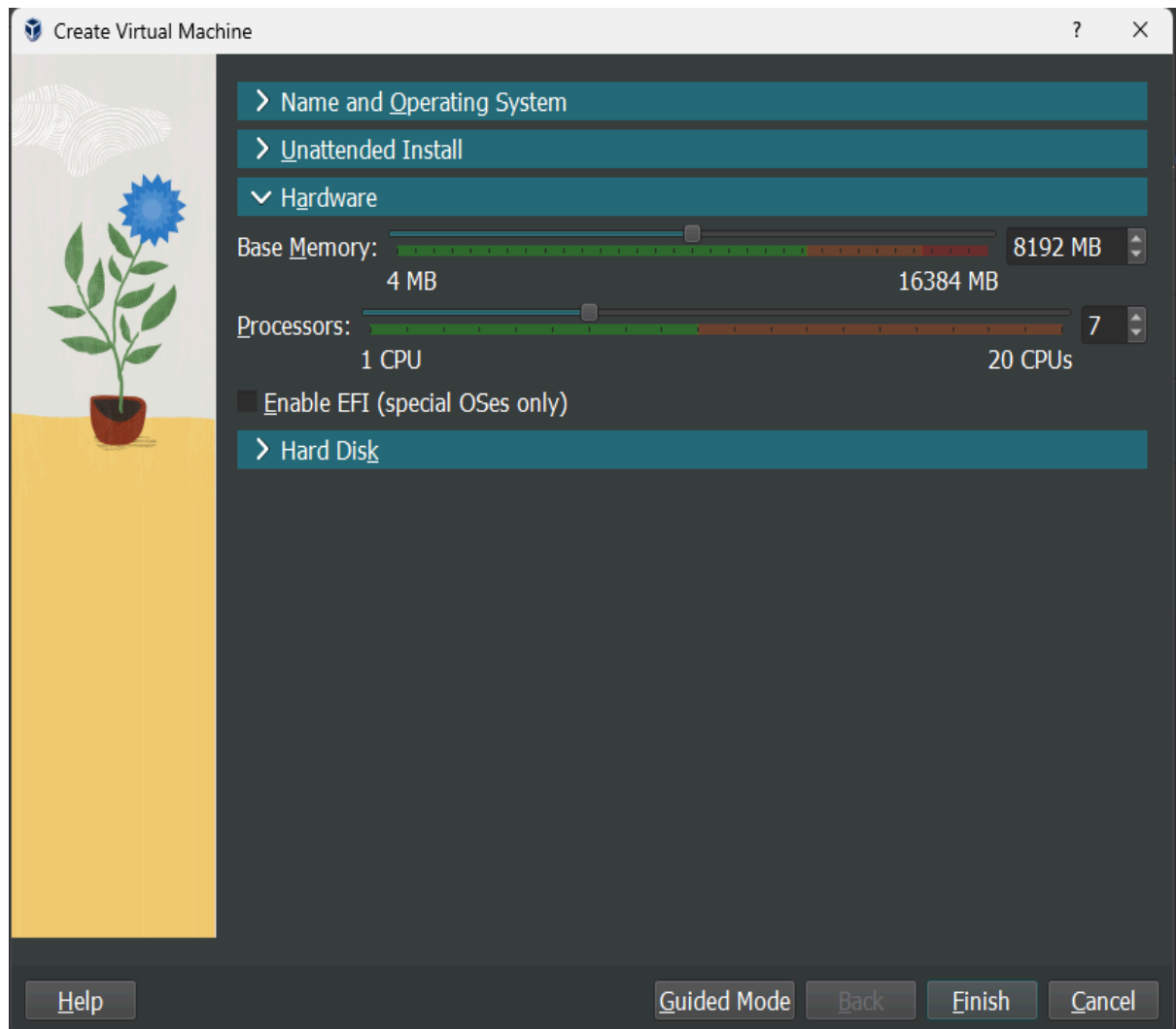
Download the version that matches your operating system (in my case, 64-bit). If you prefer, the torrent download option is also available.

Step 2:-

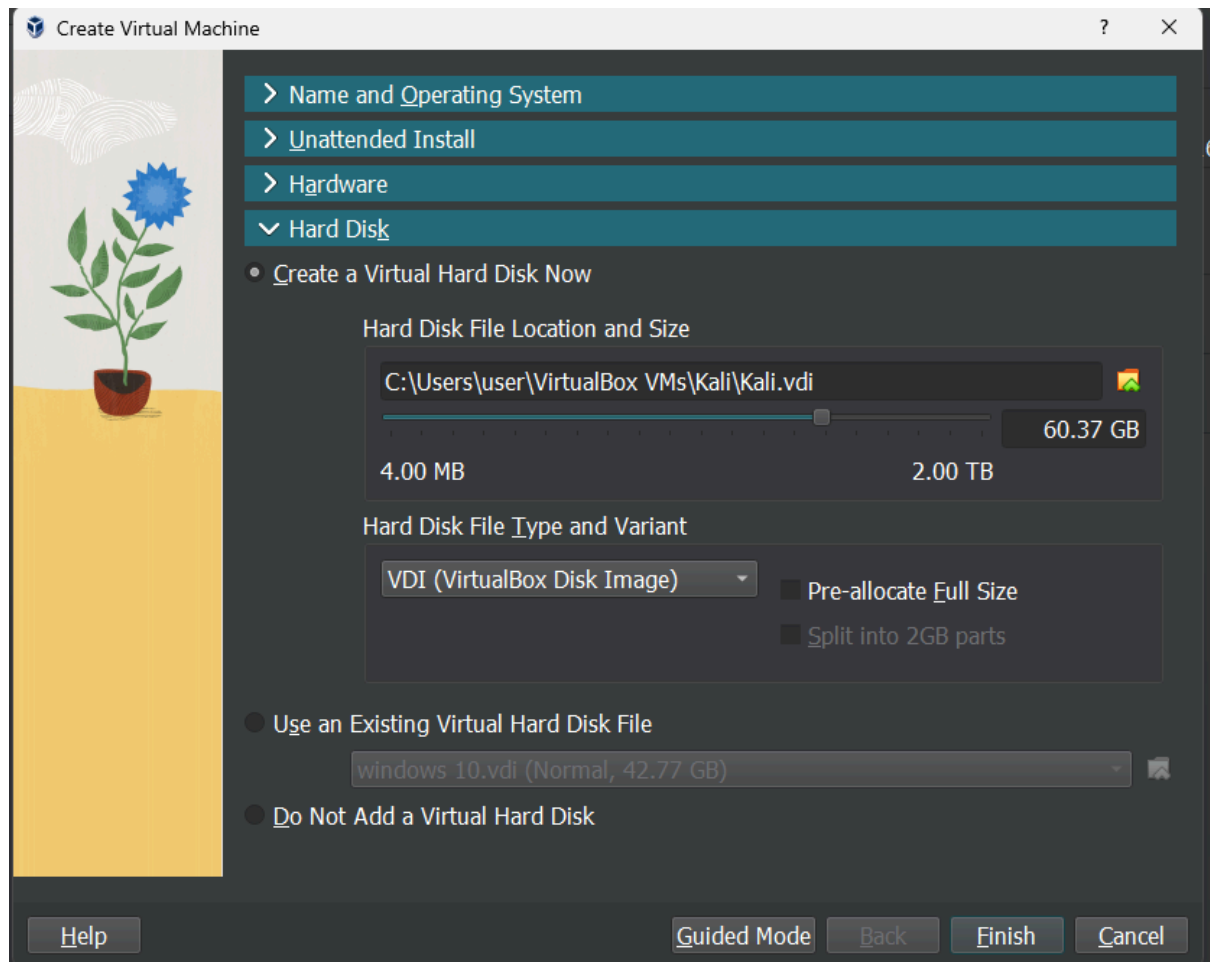
→ Create a new VM in VirtualBox or VMware



→ Select the correct type and version for Kali. I chose 'Linux' as the type and 'Debian 12 Bookworm (64-bit)' as the version because Kali is based on Debian



→ Now allocate the base memory (RAM) and processors according to your requirements.



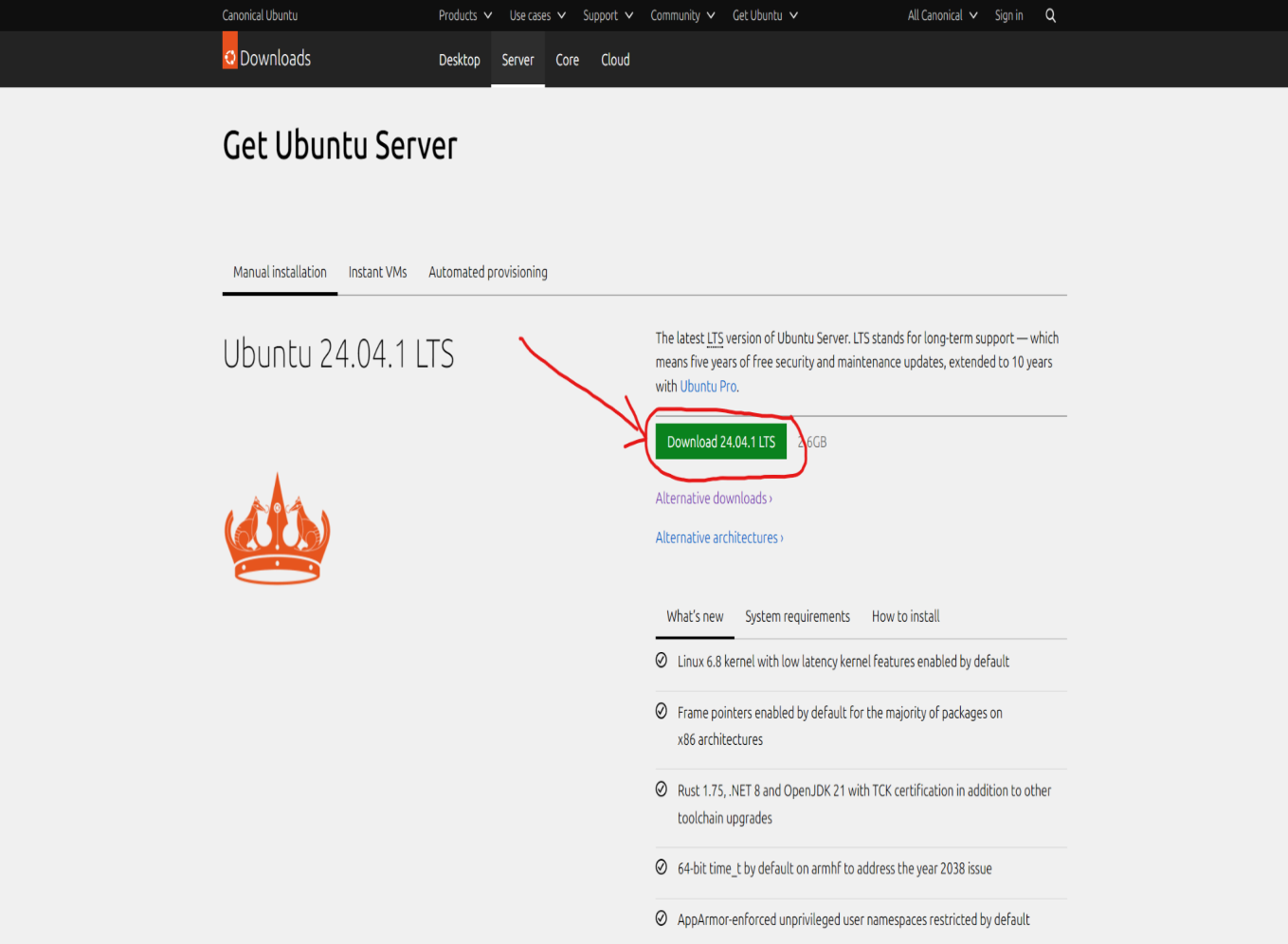
→Finally, allocate the disk space for Kali Linux based on your needs, add the ISO file, and install Kali on VirtualBox.

Target VM: Ubuntu Server(setup)

Step1:-

→Download ubuntu server

from:<https://ubuntu.com/download/server>

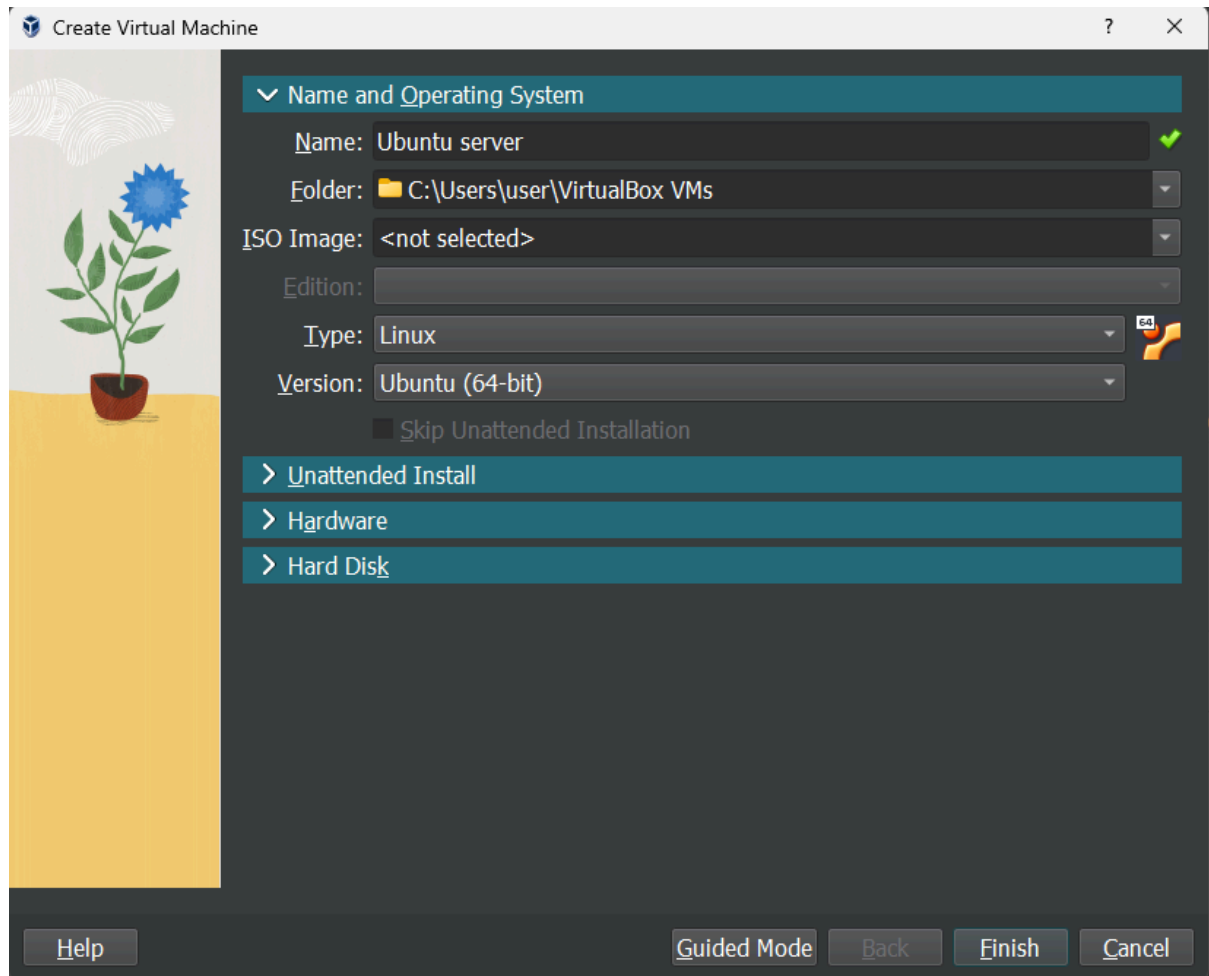


The screenshot shows the Ubuntu 24.04.1 LTS download page for the Server edition. The page has a dark header with navigation links: Canonical Ubuntu, Products, Use cases, Support, Community, Get Ubuntu, All Canonical, Sign in, and a search icon. Below the header, there's a sub-header 'Get Ubuntu Server' with tabs for Manual installation, Instant VMs, and Automated provisioning. The main content area features the Ubuntu logo, the version 'Ubuntu 24.04.1 LTS', and a green 'Download 24.04.1 LTS' button with a red circle and arrow highlighting it. To the right of the button, it says '2.6GB'. Below the button are links for 'Alternative downloads' and 'Alternative architectures'. At the bottom, there's a 'What's new' section with a list of updates: Linux 6.8 kernel, Frame pointers, Rust 1.75, .NET 8, OpenJDK 21, 64-bit time_t, and AppArmor.

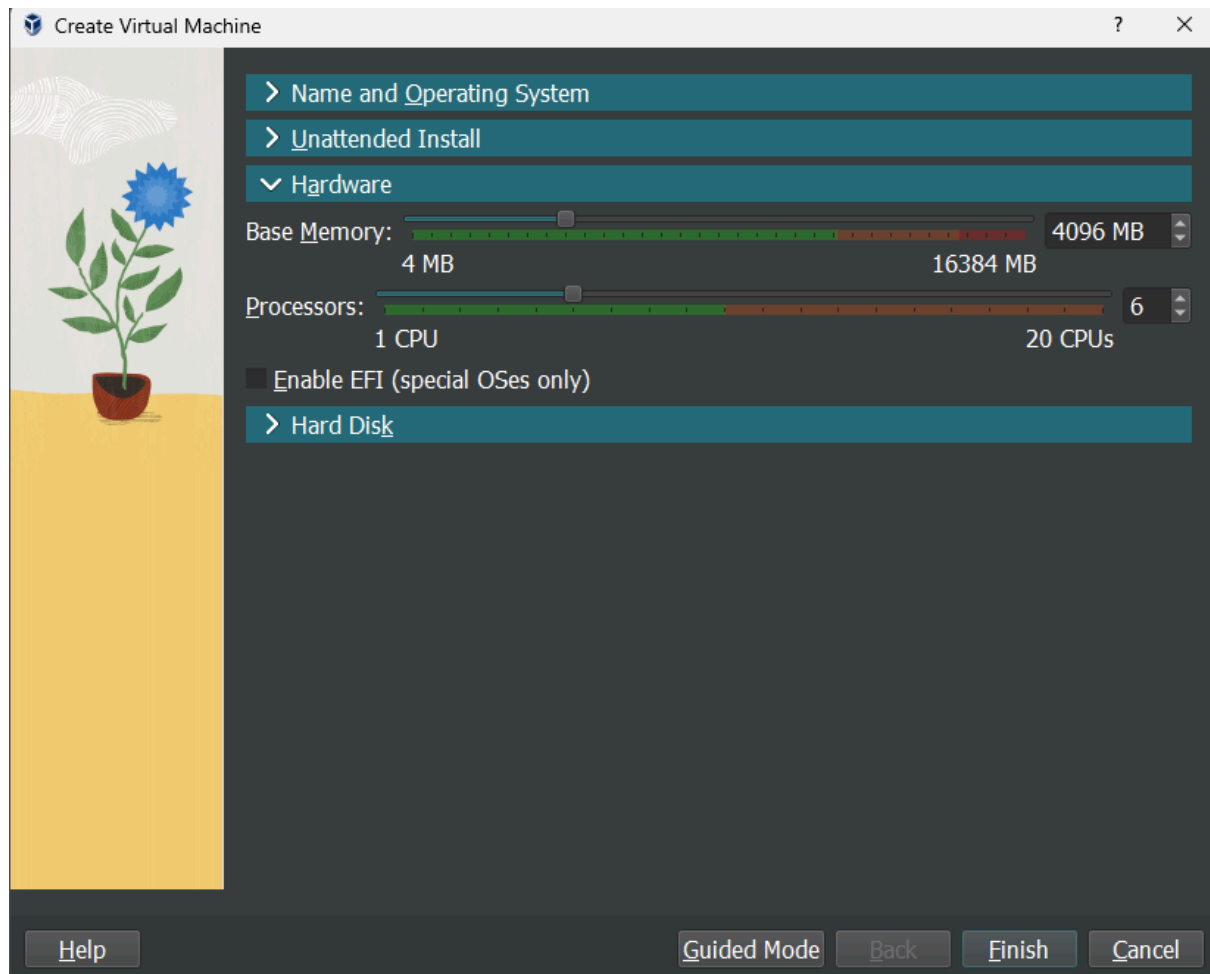
→Download the ISO file for Ubuntu Server, or if you prefer a different method, click on 'Alternative Downloads' for the torrent option.

Step 2:-

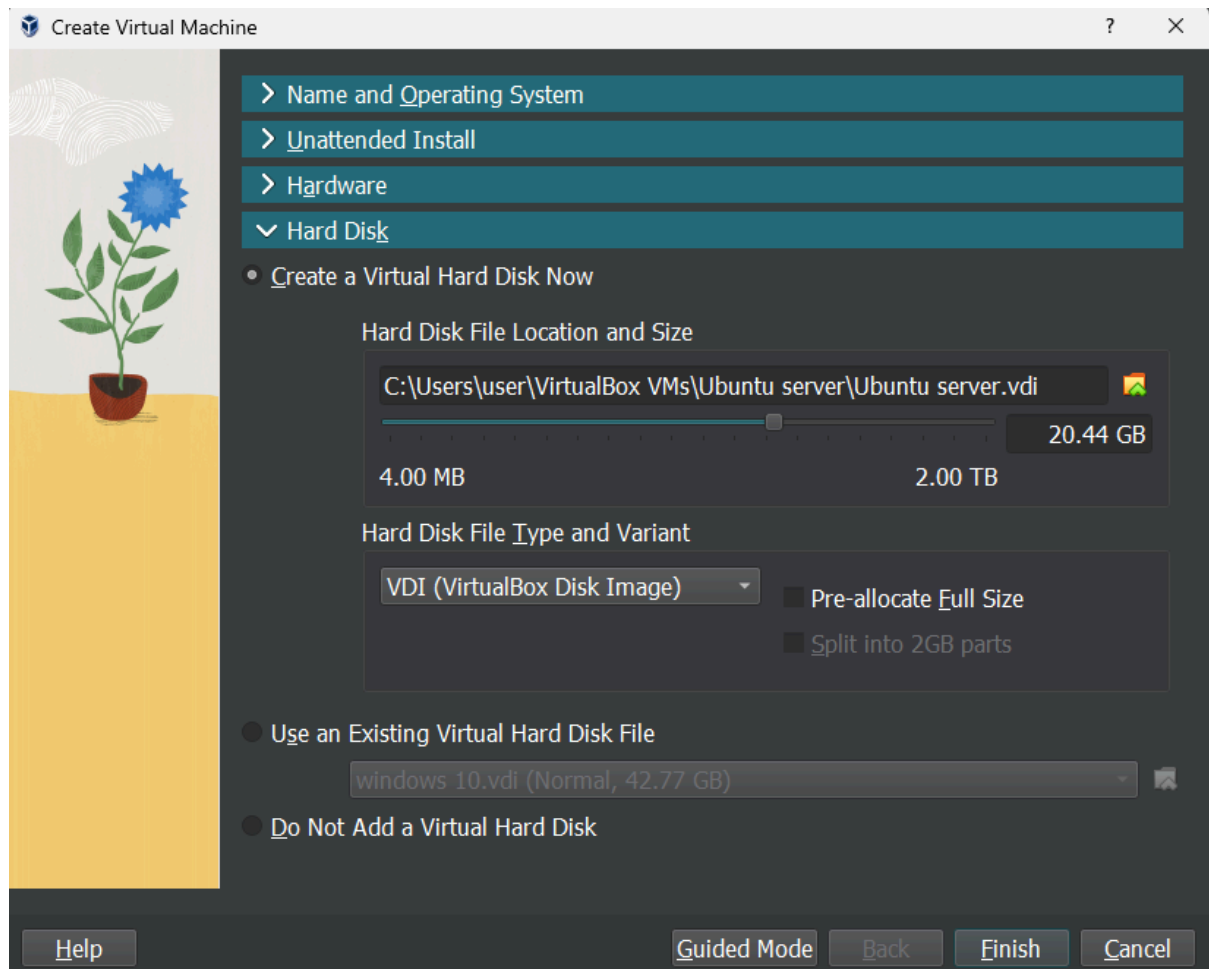
→ create a new VM for ubuntu server in virtualbox



→ Select 'Linux' as the type and 'Ubuntu (64-bit)' as the version—in my case, for Ubuntu Server.

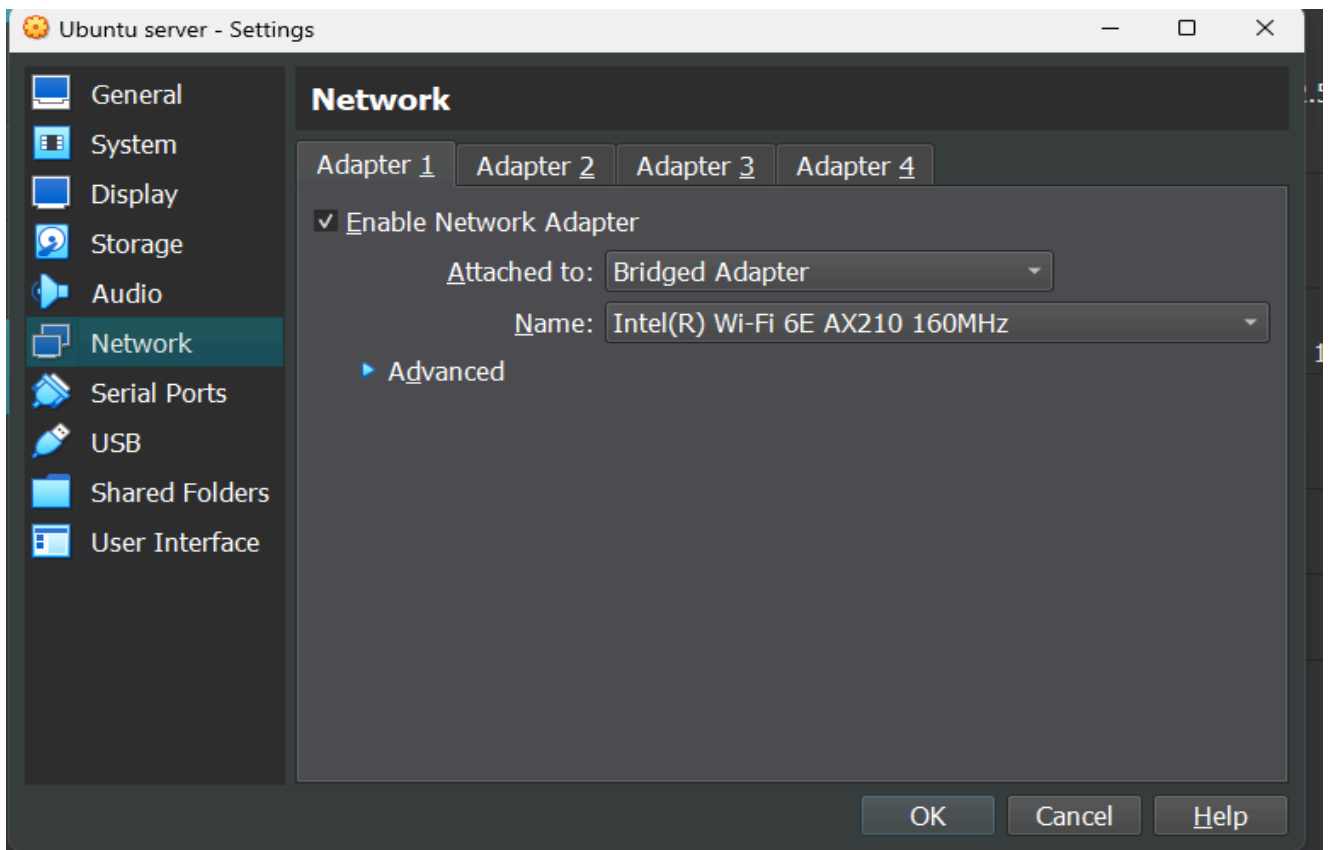
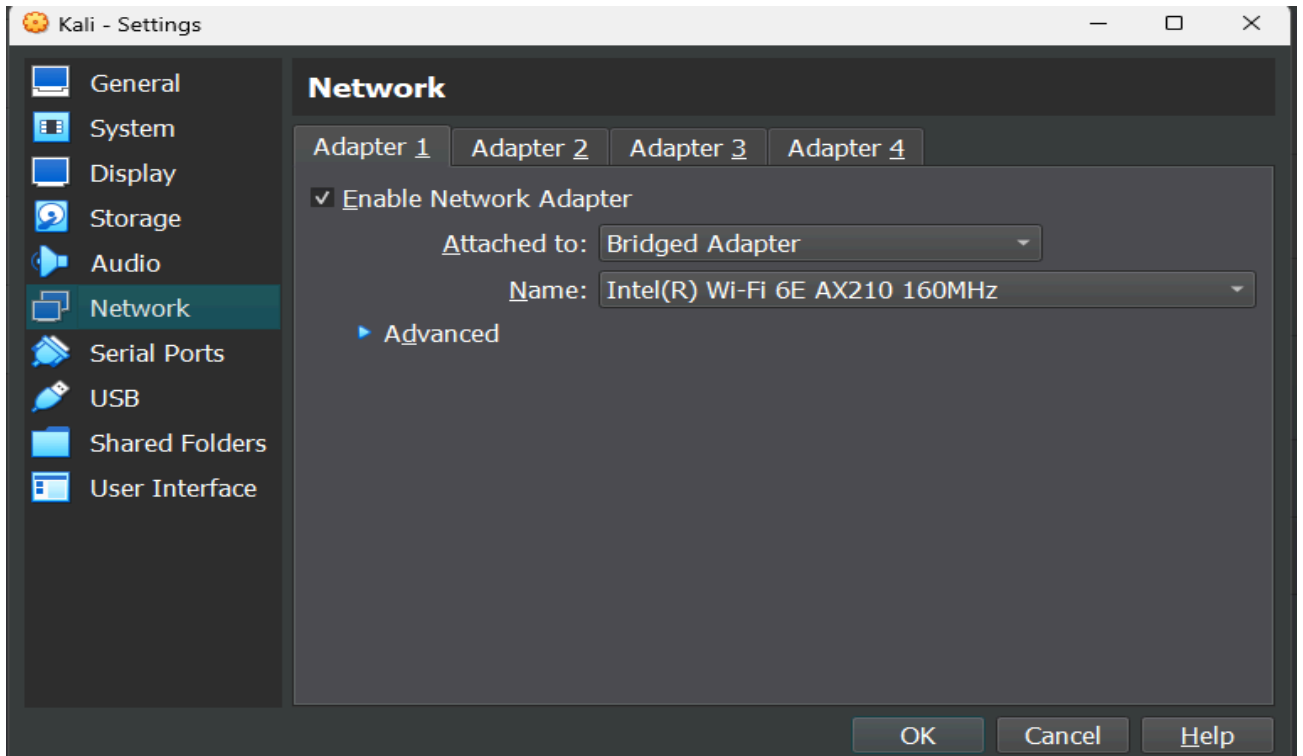


→ **Now allocate the base memory (RAM) and processors according to your preferences and system requirements.**



→ **Finally, allocate the disk space for Ubuntu Server according to your needs.**

Network Configuration



→Both the Ubuntu Server and Kali Linux should be on the same network. In my case, this ensures they can communicate effectively.

Network connectivity

```
No VM guests are running outdated hypervisor (qemu) binaries on this host.
server123@server:~$ ifconfig
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.7 netmask 255.255.255.0 broadcast 192.168.1.255
    inet6 fe80::a00:27ff:fe00:34ef prefixlen 64 scopeid 0x20<link>
    inet6 2402::a00:162:a1fc:a00:27ff:fe00:34ef prefixlen 64 scopeid 0x0<global>
    ether 08:00:27:00:34:ef txqueuelen 1000 (Ethernet)
    RX packets 477 bytes 530654 (530.0 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 280 bytes 23452 (23.4 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 118 bytes 9479 (9.4 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 118 bytes 9479 (9.4 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

server123@server:~$
```

```
moonsec@kali: ~
File Actions Edit View Help

(moonsec@kali)-[~]
$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.11 netmask 255.255.255.0 broadcast 192.168.1.255
    inet6 2402::a00:162:a1fc:a00:27ff:fe81:35ab prefixlen 64 scopeid 0x0<global>
    inet6 2402::a00:162:a1fc:af0f:81c:579d:2b36 prefixlen 64 scopeid 0x0<global>
    inet6 fe80::a00:27ff:fe81:35ab prefixlen 64 scopeid 0x20<link>
    ether 08:00:27:81:35:ab txqueuelen 1000 (Ethernet)
    RX packets 47 bytes 6206 (6.0 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 41 bytes 8828 (8.6 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 8 bytes 480 (480.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8 bytes 480 (480.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

(moonsec@kali)-[~]
$
```

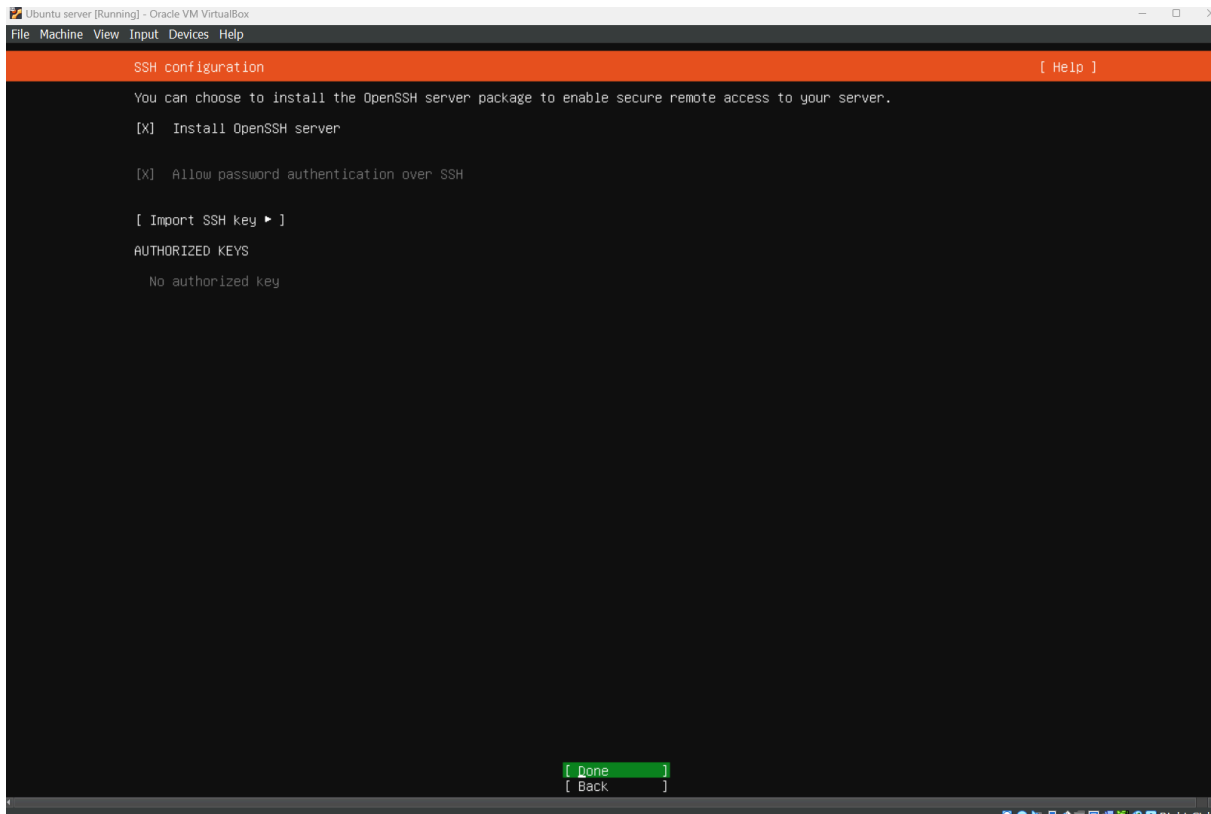
→ To check the IP address for both Ubuntu and Kali Linux, use the command `ifconfig`. This command will display network interface details, including the IP address, subnet mask, and other relevant network information for each interface. Make sure your network interfaces are properly configured before running the command.

```
(moonsec@kali)-[~]
$ ping 192.168.1.7
PING 192.168.1.7 (192.168.1.7) 56(84) bytes of data.
64 bytes from 192.168.1.7: icmp_seq=1 ttl=64 time=0.810 ms
64 bytes from 192.168.1.7: icmp_seq=2 ttl=64 time=2.92 ms
64 bytes from 192.168.1.7: icmp_seq=3 ttl=64 time=1.09 ms
64 bytes from 192.168.1.7: icmp_seq=4 ttl=64 time=0.407 ms
64 bytes from 192.168.1.7: icmp_seq=5 ttl=64 time=1.13 ms
64 bytes from 192.168.1.7: icmp_seq=6 ttl=64 time=1.95 ms
^Z
zsh: suspended ping 192.168.1.7

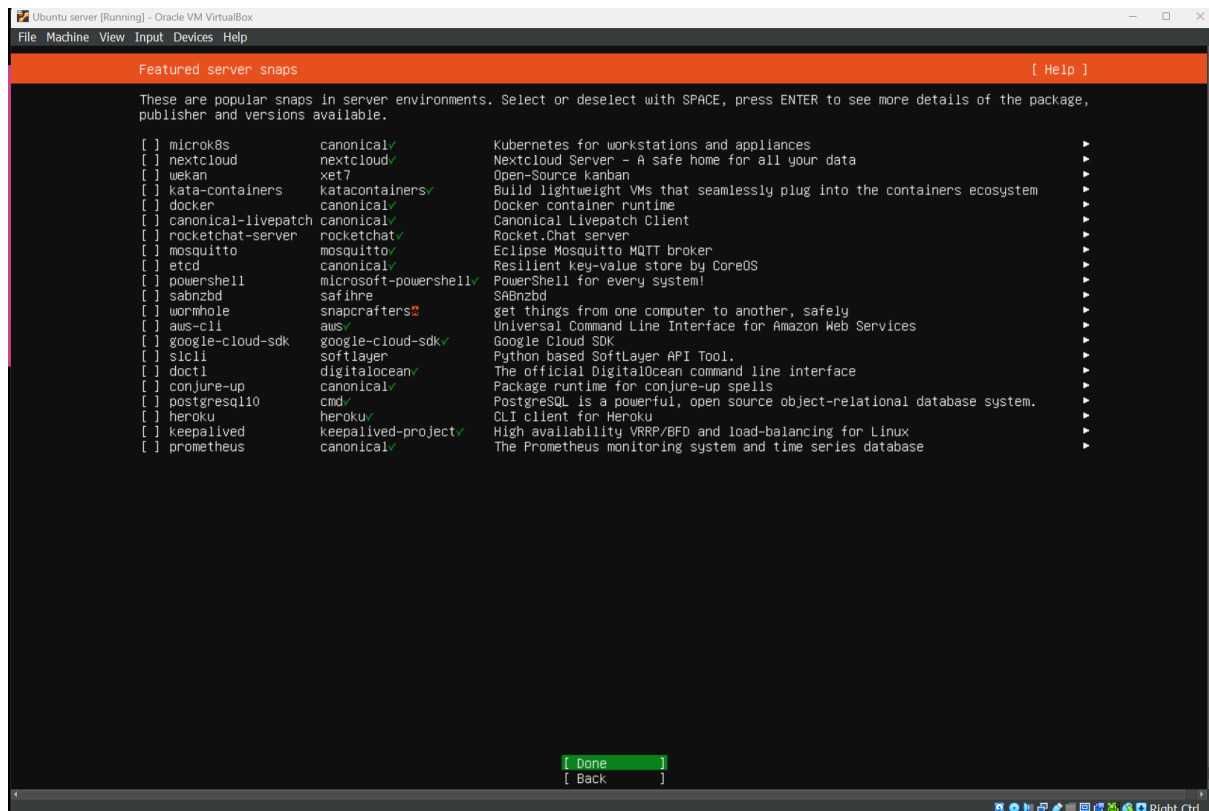
(moonsec@kali)-[~]
$ |
```

→ Now, to finally check the connectivity between the Ubuntu Server and Kali Linux, use the `ping` command. This will test if both systems are able to communicate with each other over the network. Open the terminal in either Ubuntu Server or Kali Linux and type `ping [IP address of the other machine]`. If the connection is successful, you will see replies showing the time it takes for each packet to travel between the two systems. This confirms that both Ubuntu Server and Kali Linux are on the same network and are able to communicate properly.

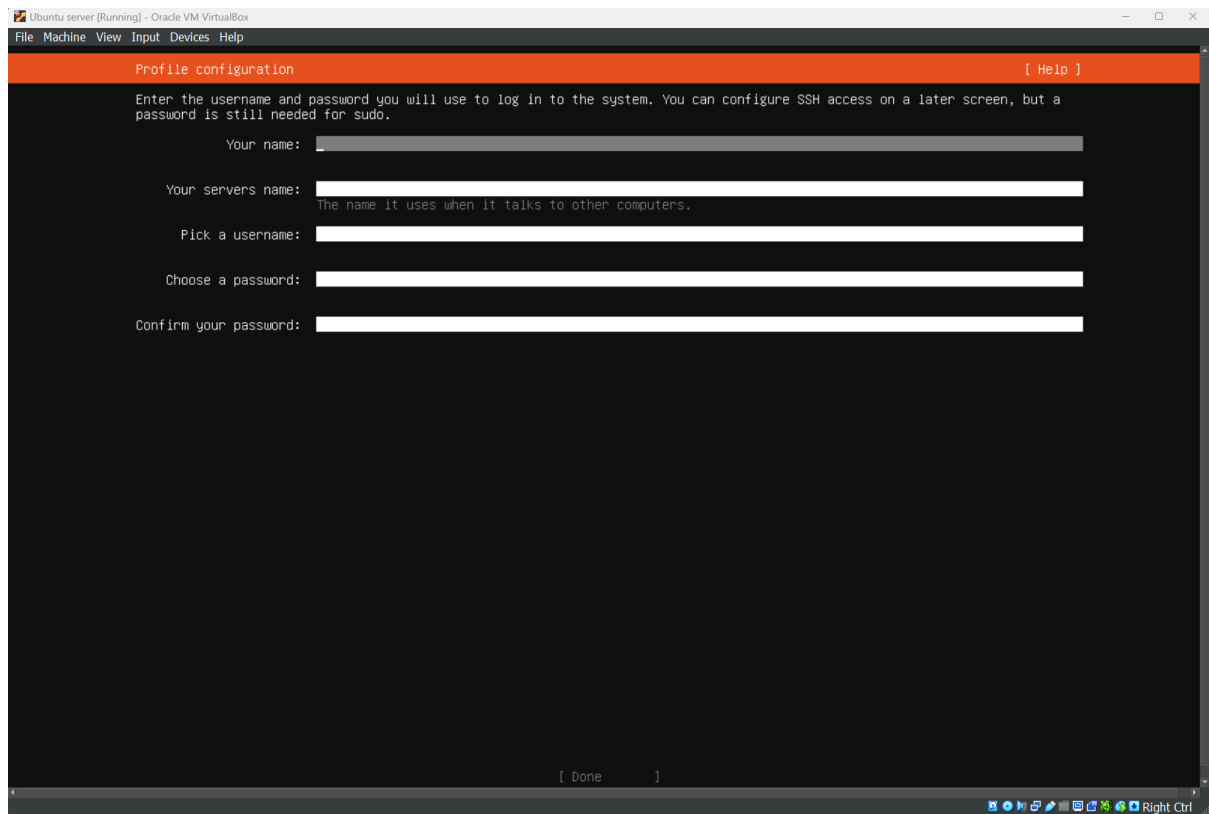
Installing and Configuring OpenSSH on Ubuntu Server



→ **When installing Ubuntu Server, it will prompt you to install OpenSSH. Press 'Enter' to select and install the OpenSSH server.**



→ To add additional features to the server, select the desired features and press 'Enter' based on your needs and requirements.



→ **Lastly, you will need to enter your full name and provide a name for the server. After that, you will be prompted to choose a username, which will be used to log in to the system. Finally, create a secure password for the account to complete the user setup process.**

→ **After successfully installing Ubuntu Server, the next step is to install Snort and the required packages. This will allow you to set up Snort for network intrusion detection and monitoring. Ensure that all necessary dependencies are installed for Snort to function properly.**

Installing snort on ubuntu server

```
server123@server123:~$ sudo apt-get update
[sudo] password for server123:
Hit:1 http://in.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://in.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://in.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Reading package lists... Done
server123@server123:~$
```

→ Before installing Snort, we need to update the Ubuntu Server by running the command: **sudo apt-get update**. This ensures that all the necessary packages and repositories are up to date and ready for the installation.

```
server123@server123:~$ sudo apt-get install snort -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
snort is already the newest version (2.9.20-0+deb11u1ubuntu1).
0 upgraded, 0 newly installed, 0 to remove and 31 not upgraded.
server123@server123:~$
```

→ To install the Snort package on Ubuntu Server, type the following command: **sudo apt-get install snort-y**. This will automatically install Snort along with its required dependencies.

Snort configuration

```
server123@server123:~$ ls -al /etc/snort
total 368
drwxr-xr-x  3 root root   4096 Sep 20 10:43 .
drwxr-xr-x 110 root root   4096 Sep 20 10:43 ..
-rw-r--r--  1 root root   1281 Apr 20 2022 attribute_table.dtd
-rw-r--r--  1 root root   3757 Apr 20 2022 classification.config
-rw-r--r--  1 root root  82469 Apr 19 12:32 community-sid-msg.map
-rw-r--r--  1 root root  23654 Apr 20 2022 file_magic.conf
-rw-r--r--  1 root root  33339 Apr 20 2022 gen-msg.map
-rw-r--r--  1 root root    687 Apr 20 2022 reference.config
drwxr-xr-x  2 root root   4096 Sep 20 10:43 rules
-rw-r-----  1 root snort 29773 Apr 19 12:32 snort.conf
-rw-----  1 root root    806 Sep 20 10:43 snort.debian.conf
-rw-r--r--  1 root root   2335 Apr 20 2022 threshold.conf
-rw-r--r--  1 root root 160606 Apr 20 2022 unicode.map
server123@server123:~$ _
```

→ To view the Snort configuration files, enter the command `ls -al /etc/snort`. This directory contains all the configuration files related to Snort.

```
server123@server123:~$ sudo vim /etc/snort/snort.conf_
```

→ To view the Snort configuration file, enter the command `sudo vim /etc/snort/snort.conf`.

```

#-----
# VRT Rule Packages Snort.conf
#
# For more information visit us at:
#   http://www.snort.org           Snort Website
#   http://vrt-blog.snort.org/     Sourcefire VRT Blog
#
# Mailing list Contact:   snort-users@lists.snort.org
# False Positive reports: fp@sourcefire.com
# Snort bugs:            bugs@snort.org
#
# Compatible with Snort Versions:
# VERSIONS : 2.9.20
#
# Snort build options:
# OPTIONS : --enable-gre --enable-mpls --enable-targetbased --enable-ppm --enable-perfprofiling --enable-zlib --enable-active-response --enable-normalizer -
# -enable-reload --enable-react --enable-flexresp3
#
# Additional information:
# This configuration file enables active response, to run snort in
# test mode -T you are required to supply an interface -i <interface>
# or test mode will fail to fully validate the configuration and
# exit with a FATAL error
#-----
#####
# This file contains a sample snort configuration.
# You should take the following steps to create your own custom configuration:
#
# 1) Set the network variables.
# 2) Configure the decoder
# 3) Configure the base detection engine
# 4) Configure dynamic loaded libraries
# 5) Configure preprocessors
# 6) Configure output plugins
# 7) Customize your rule set
# 8) Customize preprocessor and decoder rule set
# 9) Customize shared object rule set
#####
# Step #0: (Debian specific) Create a configuration
#           for a specific interface
#####
# If you want to run Snort in Debian using different
# instances each handling a different interface and
# a different configuration you can copy this file to
# /etc/snort/snort.$interface.conf (where '$interface' is the name of your
# interface)
#-----

```

→ This is the most important section of the Snort configuration file, where you can customise the rule set, set network variables, and configure plugins and many more.

```

#####
# Step #1: Set the network variables. For more information, see README.variables
#####
# Setup the network addresses you are protecting
#
# Note to Debian users: this value is overridden when starting
# up the Snort daemon through the init.d script by the
# value of DEBIAN_SNORT_HOME_NET s defined in the
# /etc/snort/snort.debian.conf configuration file
#
ipvar HOME_NET any
#
# Set up the external network addresses. Leave as "any" in most situations
ipvar EXTERNAL_NET any
# If HOME_NET is defined as something other than "any", alternative, you can
# use this definition if you do not want to detect attacks from your internal
# IP addresses:
#ipvar EXTERNAL_NET !$HOME_NET
#
# List of DNS servers on your network
ipvar DNS_SERVERS $HOME_NET
#
# List of SMTP servers on your network
ipvar SMTP_SERVERS $HOME_NET
#
# List of web servers on your network
ipvar HTTP_SERVERS $HOME_NET

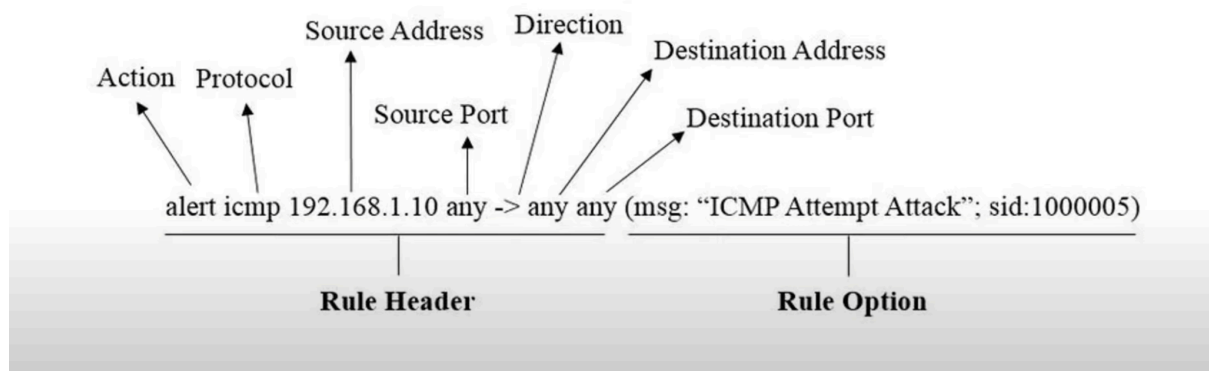
```

→ This is the section where we specify the actual network that we want to monitor. By defining the network parameters here, we ensure that Snort focuses on the correct traffic and alerts us to any suspicious activities occurring within that specified network.

```
64 #
65 ipvar HOME_NET 192.168.1.0/24
66
67 # Set up the external network addresses. Leave as "any" in most situations
68 ipvar EXTERNAL_NET any
69 # If HOME_NET is defined as something other than "any", alternative, you can
70 # use this definition if you do not want to detect attacks from your internal
71 # IP addresses:
72 #ipvar EXTERNAL_NET !$HOME_NET
```

→ Now, we need to add our network subnet in the `HOME_NET` line. In my case, it's set to `192.168.1.0/24`. Be sure to specify your own network subnet accordingly and leave `EXTERNAL_NET` set to 'any' as it is, to allow monitoring of all external traffic.

Setting Up Dashboard Rules for Snort Alerts



→ The above picture shows the syntax for the Snort rule that we are going to set.

```
server123@server123:~$ sudo vim /etc/snort/rules/local.rules_
```

→ To view and edit the rules, navigate to the rules file using the command `sudo vim /etc/snort/rules/local.rules`.

SNORPY

A Web Based Snort Rule Creator / Maker for Building Simple Snort Rules

»

TCP

☐ ACK
 ☐ SYN
 ☐ PSH
 ☐ RST
 ☐ FIN
 ☐ URG
 ☐ +
 ☐ *
 ☐

```

alert tcp any 21 -> $HOME_NET any ( msg:"ET ATTACK_RESPONSE Hostile FTP Server Banner \(\StnyFtpd\)"; flow:from_server,established;
content:"220|20|StnyFtpd|20|0wns|20|j0"; offset: 0; nocase; reference:url,doc.emergingthreats.net/bin/view/Main/2002809; classtype:trojan-activity;
sid:2002809; rev:5; )

```

→ We can also use Snorpy to automatically create rules for Snort based on specific requirements; however, for now, we will write the rules directly on the command line.

Testing and Results

Rule 1(icmp)

```
1 # $Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $
2 # -----
3 # LOCAL RULES
4 # -----
5 # This file intentionally does not come with signatures. Put your local
6 # additions here.
7
8 alert icmp any any -> $HOME_NET any (msg:"ICMP Ping Detected"; sid:100001; rev:1;) _
```

→ After navigating to the local rules file of Snort, we will write our first rule, which is an ICMP alert. The command for this rule is:

```
alert icmp any any -> $HOME_NET any (msg:"ICMP
Ping Detected"; sid:100001; rev:1;).
```

```
server123@server123:~$ sudo snort -q -l /var/log/snort -i enp0s3 -A console -c /etc/snort/snort.conf
```

→ The above command is used to run Snort. To learn more about the syntax and usage, you can type `man snort` in the terminal.

```
(moonsec@kali)-[~]
$ ping 192.168.1.10
PING 192.168.1.10 (192.168.1.10) 56(84) bytes of data.
64 bytes from 192.168.1.10: icmp_seq=1 ttl=64 time=43.6 ms
64 bytes from 192.168.1.10: icmp_seq=2 ttl=64 time=5.61 ms
64 bytes from 192.168.1.10: icmp_seq=3 ttl=64 time=0.578 ms
64 bytes from 192.168.1.10: icmp_seq=4 ttl=64 time=0.784 ms
64 bytes from 192.168.1.10: icmp_seq=5 ttl=64 time=8.14 ms
```

→ Now, we will try to ping from the attacker machine to see if Snort detects the ping request.

```
/etc/snort/snort.conf" 790L, 30057B written
server123@server123:~$ sudo snort -q -l /var/log/snort -i enp0s3 -A console -c /etc/snort/snort.conf
09/20-12:30:52.395494  [**] [1:100001:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 192.168.1.11 -> 192.168.1.10
09/20-12:30:52.395519  [**] [1:100001:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 192.168.1.10 -> 192.168.1.11
09/20-12:30:53.423762  [**] [1:100001:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 192.168.1.11 -> 192.168.1.10
09/20-12:30:53.423790  [**] [1:100001:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 192.168.1.10 -> 192.168.1.11
09/20-12:30:54.453628  [**] [1:100001:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 192.168.1.11 -> 192.168.1.10
09/20-12:30:54.453649  [**] [1:100001:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 192.168.1.10 -> 192.168.1.11
09/20-12:30:55.462353  [**] [1:100001:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 192.168.1.11 -> 192.168.1.10
09/20-12:30:55.462400  [**] [1:100001:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 192.168.1.10 -> 192.168.1.11
09/20-12:30:56.464442  [**] [1:100001:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 192.168.1.11 -> 192.168.1.10
09/20-12:30:56.464466  [**] [1:100001:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 192.168.1.10 -> 192.168.1.11
09/20-12:30:57.490571  [**] [1:100001:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 192.168.1.11 -> 192.168.1.10
09/20-12:30:57.490615  [**] [1:100001:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 192.168.1.10 -> 192.168.1.11
09/20-12:30:58.497839  [**] [1:100001:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 192.168.1.11 -> 192.168.1.10
09/20-12:30:58.497895  [**] [1:100001:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 192.168.1.10 -> 192.168.1.11
09/20-12:30:59.535860  [**] [1:100001:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 192.168.1.11 -> 192.168.1.10
09/20-12:30:59.535938  [**] [1:100001:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 192.168.1.10 -> 192.168.1.11
09/20-12:31:00.562459  [**] [1:100001:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 192.168.1.11 -> 192.168.1.10
09/20-12:31:00.562478  [**] [1:100001:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 192.168.1.10 -> 192.168.1.11
09/20-12:31:01.740080  [**] [1:100001:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 192.168.1.11 -> 192.168.1.10
09/20-12:31:01.740103  [**] [1:100001:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 192.168.1.10 -> 192.168.1.11
09/20-12:31:02.912115  [**] [1:100001:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 192.168.1.11 -> 192.168.1.10
09/20-12:31:02.912134  [**] [1:100001:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 192.168.1.10 -> 192.168.1.11
09/20-12:31:04.068512  [**] [1:100001:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 192.168.1.11 -> 192.168.1.10
09/20-12:31:04.068557  [**] [1:100001:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 192.168.1.10 -> 192.168.1.11
09/20-12:31:05.367029  [**] [1:100001:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 192.168.1.11 -> 192.168.1.10
```

→ Bingo! Snort is working perfectly fine; it has detected the attacker's ping request and alerted us immediately.

Rule 2(ssh) Brute force Attack

```
1 # $Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $
2 # -----
3 # LOCAL RULES
4 # -----
5 # This file intentionally does not come with signatures. Put your local
6 # additions here.
7
8 alert icmp any any -> $HOME_NET any (msg:"ICMP Ping Detected" ; sid:100001 ; rev:1;)
9 alert tcp any any -> $HOME_NET 22 (msg:"SSH Authentication Attempt" ; sid:100002 ; rev:1;)
```

→ Now we need to add a new rule for SSH detection. As you can see, I have changed the port number from 'any' to '22' because SSH runs on port 22. This way, Snort will specifically detect traffic on this designated port for this rule.

```
(moonsec@kali)-[~]
└─$ hydra -l root -P /usr/share/wordlists/rockyou.txt 192.168.1.10 -t 5 ssh
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organization
s, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2024-09-20 18:37:37
[DATA] max 5 tasks per 1 server, overall 5 tasks, 14344399 login tries (l:1/p:14344399), ~2868880 tries per task
[DATA] attacking ssh://192.168.1.10:22/
```

→ Now, from the attacker machine, we will attempt a brute force attack on SSH on the Ubuntu Server where we have installed Snort for monitoring. Let's check if it detects the login attempts.

```

09/20-13:08:50.676871 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54186 -> 192.168.1.10:22
09/20-13:08:50.676871 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54206 -> 192.168.1.10:22
09/20-13:08:50.676871 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54172 -> 192.168.1.10:22
09/20-13:08:50.676871 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54188 -> 192.168.1.10:22
09/20-13:08:50.678709 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54206 -> 192.168.1.10:22
09/20-13:08:50.678952 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54172 -> 192.168.1.10:22
09/20-13:08:50.679493 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54188 -> 192.168.1.10:22
09/20-13:08:50.864064 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54190 -> 192.168.1.10:22
09/20-13:08:50.991194 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54172 -> 192.168.1.10:22
09/20-13:08:50.991194 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54186 -> 192.168.1.10:22
09/20-13:08:50.991195 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54206 -> 192.168.1.10:22
09/20-13:08:51.001640 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54188 -> 192.168.1.10:22
09/20-13:08:52.931789 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54188 -> 192.168.1.10:22
09/20-13:08:52.931789 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54190 -> 192.168.1.10:22
09/20-13:08:53.070494 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54172 -> 192.168.1.10:22
09/20-13:08:53.085370 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54206 -> 192.168.1.10:22
09/20-13:08:53.110583 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54186 -> 192.168.1.10:22
09/20-13:08:55.478687 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54190 -> 192.168.1.10:22
09/20-13:08:57.555689 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54206 -> 192.168.1.10:22
09/20-13:08:57.590972 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54186 -> 192.168.1.10:22
09/20-13:08:57.662873 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54172 -> 192.168.1.10:22
09/20-13:08:57.780779 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54190 -> 192.168.1.10:22
09/20-13:09:00.024189 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54190 -> 192.168.1.10:22
09/20-13:09:00.461919 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54206 -> 192.168.1.10:22
09/20-13:09:01.852841 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54188 -> 192.168.1.10:22
09/20-13:09:03.010283 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54186 -> 192.168.1.10:22
09/20-13:09:03.024500 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54172 -> 192.168.1.10:22
09/20-13:09:04.928234 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54206 -> 192.168.1.10:22
09/20-13:09:05.245835 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54186 -> 192.168.1.10:22
09/20-13:09:05.260042 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54172 -> 192.168.1.10:22
09/20-13:09:05.387340 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54190 -> 192.168.1.10:22
09/20-13:09:06.326541 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54188 -> 192.168.1.10:22
09/20-13:09:07.187385 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54206 -> 192.168.1.10:22
09/20-13:09:09.420275 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54206 -> 192.168.1.10:22
09/20-13:09:09.422360 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54206 -> 192.168.1.10:22
09/20-13:09:09.707049 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54186 -> 192.168.1.10:22
09/20-13:09:09.723574 [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54172 -> 192.168.1.10:22

```

→ Once again, Snort has successfully identified the SSH login attempts and alerted us immediately. This demonstrates its effectiveness in monitoring network traffic and detecting unauthorised access attempts. The alerts provide crucial information about the nature of the attack, allowing us to take appropriate actions to enhance security.

```

(moonsec@kali)-[~]
$ nmap -P 22 192.168.1.10
Warning: You are not root -- using TCP pingscan rather than ICMP
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-09-20 18:51 IST
Nmap scan report for 192.168.1.10 (192.168.1.10)
Host is up (0.00048s latency).
Not shown: 999 closed tcp ports (conn-refused)
PORT      STATE SERVICE
22/tcp    open  ssh

```

→ Let's try performing an SSH login attempt using Nmap and check the results. This will allow us to see how Snort responds to scanning activities related to SSH and whether it effectively detects the attempts.

```
server123@server123:~$ sudo snort -q -l /var/log/snort -i enp0s3 -A console -c /etc/snort/snort.conf
09/20-13:22:31.503032  [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54584 -> 192.168.1.10:22
09/20-13:22:31.522414  [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54584 -> 192.168.1.10:22
09/20-13:22:31.534086  [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54584 -> 192.168.1.10:22
09/20-13:22:31.541902  [**] [1:100002:1] SSH Authentication Attempt [**] [Priority: 0] {TCP} 192.168.1.11:54584 -> 192.168.1.10:22
```

→ **Even when attempting to log in with Nmap, Snort is able to detect the requests coming from the attacker machine. This highlights Snort's capability to monitor and alert us about potentially malicious scanning activities on the network.**

Conclusion

This project successfully demonstrated the setup and configuration of a network security lab using Snort IDS on an Ubuntu Server, alongside an attack simulation using Kali Linux. By creating virtual machines in VirtualBox, we established a controlled environment where we could simulate cyber threats, specifically focusing on brute-force attacks and ICMP ping requests.

Through this hands-on experience, we gained valuable insights into the functioning of an Intrusion Detection System (IDS) and its capability to monitor network traffic for potential threats. We implemented two specific rules in Snort to detect:

- 1. ICMP Ping Requests: This rule allowed us to monitor for unsolicited ping requests, helping to identify potential reconnaissance activities.**
- 2. TCP SSH Login Attempts: By capturing SSH login attempts, we could detect brute-force attacks aimed at gaining unauthorised access to the server.**

To simulate these attacks, we utilised tools such as Nmap for scanning the SSH service and Hydra for conducting brute-force login attempts. These tools proved essential in demonstrating how attackers might exploit vulnerabilities in network services.

The flexibility of Snort allows for the addition of numerous rules to enhance monitoring and detection capabilities. While this project focused on just two rules, we could expand this setup to include various other rules tailored to specific threats or vulnerabilities.

In conclusion, this project not only provided practical experience in configuring IDS and simulating attacks but also emphasised the importance of proactive threat detection in

cybersecurity. As network environments become increasingly complex, tools like Snort will play a crucial role in safeguarding against potential cyber threats.